

2026-05-05-p1-f10-skill-system-design

Phase 1 / Feature 10 — Skill System

Date: 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

1. Goal

Add a parallel system for **agent-invoked Skills** (distinct from F09's user-invoked commands). Skills are `.helix/skills/*.md` files with YAML frontmatter (description, triggers `[]string regex, variables map[string]string, requires_isolation bool`) plus a Markdown body. The agent's main loop checks user input against every loaded skill's trigger patterns (first match wins) and substitutes the rendered body as a system-prompt instruction. Skills with `requires_isolation: true` execute in their own F04 worktree; results are surfaced back to the parent session. Both `/skills slash` and `helixcode skills cobra` surfaces expose `list/show <name>/invoke <name>/reload`.

2. Architecture

Add `internal/commands/markdown_skills.go` (same package as F09 — reuses `parseMarkdownCommand`'s frontmatter/body splitting helper and the `substRegex/tokenResolver` machinery). New `Skill` type is distinct from `MarkdownCommand` — different lifecycle (auto-invoked vs user-invoked) and different fields (`TriggerPatterns`, `RequiresIsolation`). New `SkillLoader` (mirrors F09's `MarkdownLoader`) scans `.helix/skills/` (project) and `~/.config/helixcode/skills/` (user) with `fsnotify` hot-reload. `SkillRegistry` holds compiled trigger regexes and exposes `FindMatching(input)`. Auto-invocation in the agent loop calls `FindMatching` before each LLM call; on match, the rendered skill body is injected as a system message. Isolation routes through `internal/tools/worktree/` (F04's `WorktreeManager`) — skill runs in a fresh worktree, output captured.

3. Components

3.1 New files

- `helix_code/internal/commands/markdown_skills.go` — `Skill`, `SkillRegistry`, `SkillLoader`, `parseSkillFile`, `compileTriggers`, `FindMatching`
- `helix_code/internal/commands/markdown_skills_test.go` — unit tests
- `helix_code/internal/commands/skills_watcher.go` — `fsnotify` wrapper (or extend the F09 watcher to monitor both dirs)
- `helix_code/internal/commands/skills_watcher_test.go`
- `helix_code/internal/commands/skills_command.go` — `/skills slash` command
- `helix_code/internal/commands/skills_command_test.go`
- `helix_code/cmd/cli/skills_cmd.go` — cobra subcommand

- `helix_code/cmd/cli/skills_cmd_test.go`
- `helix_code/internal/agent/skill_dispatcher.go` — auto-invocation hook called from the agent loop
- `helix_code/internal/agent/skill_dispatcher_test.go`
- `helix_code/tests/integration/skills_test.go` — `//go:build integration`
- `challenges/p1-f10-skills/CHALLENGE.md + run.sh`

3.2 Modified

- `helix_code/internal/agent/base_agent.go` (or wherever the LLM-call loop lives) — wire `skillDispatcher.Match(userInput)` before each LLM turn; on match, inject skill body as system message
- `helix_code/cmd/cli/main.go` — construct `SkillLoader + SkillRegistry + dispatcher`; register `/skills slash`; wire cobra dispatcher; pass loader to existing watcher (or start a second watcher)

3.3 Types

```

type Skill struct {
    name           string
    description     string
    body           string
    variables       map[string]string
    triggerPatterns []string           // raw strings
    triggers       []*regexp.Regexp  // compiled
    requiresIsolation bool
    sourcePath     string
}

func (s *Skill) Name() string
func (s *Skill) Description() string
func (s *Skill) Render(args []string, selection, currentFile string) (string, error)
func (s *Skill) RequiresIsolation() bool

type SkillRegistry struct {
    mu      sync.RWMutex
    skills map[string]*Skill
}

func NewSkillRegistry() *SkillRegistry
func (r *SkillRegistry) Add(s *Skill)
func (r *SkillRegistry) Remove(name string)
func (r *SkillRegistry) Get(name string) (*Skill, bool)
func (r *SkillRegistry) List() []*Skill
// FindMatching returns the first skill whose trigger patterns
// match input,
// plus the named-capture groups extracted from the match (used
// as substitution

```

```
// args via tokenResolver's ARG.name lookup).
func (r *SkillRegistry) FindMatching(input string) (*Skill,
    map[string]string, bool)

type SkillLoader struct {
    registry    *SkillRegistry
    projectDir  string
    userDir     string
    log         *zap.Logger
    mu          sync.Mutex
    loaded      map[string]string // name → source path
}

func NewSkillLoader(reg *SkillRegistry, projectDir, userDir
    string) *SkillLoader
func (l *SkillLoader) Load() error
func (l *SkillLoader) Reload() error
func (l *SkillLoader) Loaded() map[string]string
```

3.4 Frontmatter format

```
---
description: Refactor a React component
triggers:
  - "(?i)^refactor (.+) component$"
  - "(?i)^extract hook from (.+)"
variables:
  default_style: functional
requires_isolation: false
---
```

You are refactoring `{{ARG1}}`.
 Extract reusable hooks where the file count > 200 lines.
 Style: {{ARG.default_style}}.

3.5 Auto-invocation flow

```
agent main loop receives user input
├─ skillDispatcher.Match(userInput)
│   └─ registry.FindMatching(userInput)
│       └─ for each skill, for each compiled regex:
│           if regex.FindStringSubmatch(input) != nil:
│               extract named capture groups → map
│               return (skill, captures, true)
├─ if matched:
│   └─ if skill.RequiresIsolation():
│       └─ wt := worktree.Create(name="skill-<id>")
│           └─ run skill body as system message in isolated session
│           └─ surface result back, optionally merge worktree
└─ else:
```

```

└─ render skill body via tokenResolver(captures)
    inject as additional system message before LLM call
└─ proceed with normal LLM call

```

3.6 User surface

/skills: - /skills (default list) → table NAME / DESCRIPTION / TRIGGERS /
 SOURCE - /skills show <name> → render skill metadata + body - /skills invoke
 <name> [args...] → run the skill explicitly (bypassing trigger matching) - /skills
 reload → re-scan dirs

helixcode skills: list, show <name>, invoke <name> [args...], reload
 — same operations via cobra.

4. Data flow

4.1 Startup

```

main.go
└─ skillReg := NewSkillRegistry()
└─ skillLoader := NewSkillLoader(skillReg, ".helix/skills", "~/.config/h
└─ skillLoader.Load() (compiles trigger regexes; logs WARN on bad regex
└─ go fsnotifyWatch(skillLoader, both dirs) // reuse F09 watcher patter
└─ skillDispatcher := agent.NewSkillDispatcher(skillReg, worktreeMgr)
└─ baseAgent.SetSkillDispatcher(skillDispatcher)
└─ cmdRegistry.Register(commands.NewSkillsCommand(skillLoader, skillReg)
└─ rootCmd.AddCommand(newSkillsCmd(skillsCmdDeps{...}))

```

4.2 Auto-invocation

```

user message arrives at agent
└─ baseAgent.Run(input)
    └─ skill, args, matched := skillDispatcher.Match(input)
    └─ if matched:
        └─ rendered := skill.Render(args, selection, currentFile)
        └─ injectSystemMessage(rendered) // before LLM call
    └─ normal LLM call

```

4.3 Isolated invocation (requires_isolation: true)

```

skillDispatcher.MatchAndRun(skill, args, ctx)
└─ wt := worktreeMgr.Create("skill-" + skill.Name())
└─ subAgent := agent.NewSubAgent(wt.Path)
└─ result := subAgent.Run(rendered)
└─ surface result; user decides via /skills accept|discard whether to me

```

For F10 ship, the isolated path captures stdout/result; merging is a follow-up (in scope: out-of-scope for §8).

5. Error handling, edge cases

- **Bad trigger regex:** log WARN, skip the skill on Load.

- **Skill body parse error:** skip skill, log path.
- **Multiple skills match:** first registered wins (deterministic via lexicographic sort of names).
- **Capture group naming collision with positional ARG:** named captures take precedence over positional ARG_N (token resolver checks ARG.name first).
- `requires_isolation: true` **without worktree manager wired:** log WARN, fall back to in-session injection.
- **Reload while skill is mid-execution:** existing in-flight execution finishes; next call uses new registry state.

Anti-bluff (CONST-035 / §11.9)

- Challenge writes `.helix/skills/refactor-button.md` with trigger `^refactor (.+) component$`, sends user input “refactor LoginButton component”, asserts Match returns the skill + capture group “LoginButton”, asserts rendered body contains the captured name.
- Tests use real files via `t.TempDir()` — no mocks.
- Anti-bluff smoke must remain clean.

6. Testing

Unit tests: - `TestParseSkillFile_Valid` (frontmatter + triggers compile) - `TestParseSkillFile_BadRegexSkipped` - `TestSkill_Render_NamedCaptures` (regex group `(?P<name>...)` flows to `ARG.name`) - `TestSkill_Render_PositionalFromInput` (numbered args from match groups) - `TestSkillRegistry_FindMatching_FirstWins` - `TestSkillRegistry_AddRemove` - `TestSkillLoader_LoadProjectAndUser` - `TestSkillLoader_ReloadDiff` - `TestSkillLoader_BadFrontmatterSkipped` - `TestSkillsWatcher_DebouncesAndReloads` - `TestSkillsWatcher_StopsOnContextCancel` - `TestSlashSkills_List/Show/Invoke/Reload/UnknownSubcommandErrors` - `TestSkillsCmd_List/Show/Invoke` - `TestSkillDispatcher_Match_Injects` (no isolation) - `TestSkillDispatcher_Match_RequiresIsolation_RoutesToWorktree` (with mocked `WorktreeManager`)

Integration test (real fs + real agent path): - `TestSkills_AutoInvokesOnTriggerMatch` - `TestSkills_IsolationCreatesWorktree`

Challenge harness exercises `load` → `match` → `render` → `assert capture flows correctly`.

7. Cross-platform

Pure Go; `fsnotify` works on Linux/macOS/Windows. Cross-compile linux is the canonical check.

8. Out of scope (deferred)

- Result-merging from isolated worktrees back to the parent (currently isolated skills run, output is captured; merge UX is F10.5).
- Skill versioning / signature verification.
- Recursive skill invocation (skill A triggers skill B).
- Per-user skill marketplace / discovery.

9. Constitutional compliance

- §11.9: Challenge captures real Match → Render output from real .md file.
- CONST-042: skill bodies may include `{ { ENV . NAME } }`; loader does NOT log resolved values.
- CONST-043: non-force pushes to all four remotes.

10. Open questions resolved

Q	Answer
Q1: code organisation	(A) <code>internal/commands/markdown_skills.go</code> (same package as F09)
Q2: auto-invocation	(A) Single regex match per turn, first match wins
Q3: fork isolation	(A) Integrate with F04's WorktreeManager
Q4: user surface	(A) Both <code>/skills slash + helixcode</code> <code>skills cobra</code>
Q5: loader/watcher	(A) Sibling SkillLoader + dedicated watcher mirroring F09